Personal Kanban — Architecture & Codebase Reference

> *A single-user Kanban board built with React 18, Vite 5, and Supabase.*
> *Real-time sync · Drag-and-drop · WIP limits · Priority & due-date filtering*

Table of Contents

Tech Stack

Layer	Library / Tool	Version	Purpose
UI Framework	React	18.3.1	Component rendering
Build Tool	Vite	5.4.1	Dev server + bundler
Database	Supabase	2.39.3	Postgres DB + realtime WebSocket
Drag & Drop	@hello-pangea/dnd	16.6.0	Drag cards between columns
React plugin	@vitejs/plugin-react	4.3.1	Babel/JSX transform for Vite

No state management library (Redux/Zustand/Jotai). All state lives in React's built-in `useState` and `useMemo`.

Project Structure

```
personal-kanban/
├── index.html                ← HTML shell / Vite entry
├── vite.config.js            ← Vite build configuration
├── package.json              ← Dependencies & scripts
├── .env.example              ← Environment variable template
├── .gitignore
│
```

```
└── src/
    ├── main.jsx                ← React DOM mount
    ├── App.jsx                 ← Root component (state orchestrator)
    ├── App.css                 ← All styles (single stylesheet)
    ├── constants.js            ← Column definitions, colors, WIP limit
    │
    ├── lib/
    │   └── supabase.js         ← Supabase client singleton
    │
    ├── hooks/
    │   └── useCards.js         ← All card CRUD + realtime logic
    │
    └── components/
        ├── Board.jsx           ← DragDropContext wrapper + column grid
        ├── Column.jsx          ← Single column (Droppable zone)
        ├── Card.jsx            ← Single card (Draggable item)
        ├── AddCardModal.jsx    ← Modal form to create a new card
        ├── FilterBar.jsx       ← Priority + due-date filter buttons
        └── StatsBar.jsx        ← Header stats strip (total/wip/overdue/done)
```

Data Model

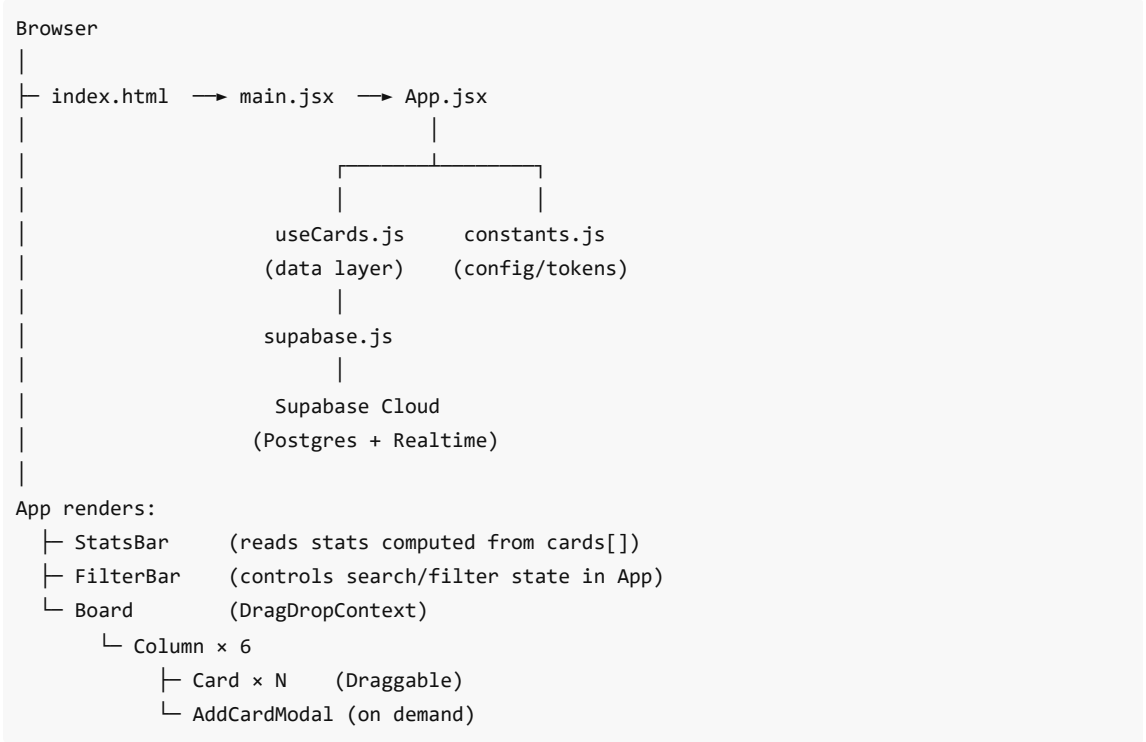
Each card is a row in the `cards` table in Supabase Postgres:

Column	Type	Notes
`id`	uuid (PK)	Auto-generated by Supabase
`title`	text	Card label, max 200 chars
`priority`	text	`'high'` | `'medium'` | `'low'`
`due_date`	date (nullable)	ISO date string `YYYY-MM-DD`
`col`	text	Column id (see columns below)
`position`	integer	Sort order within the column

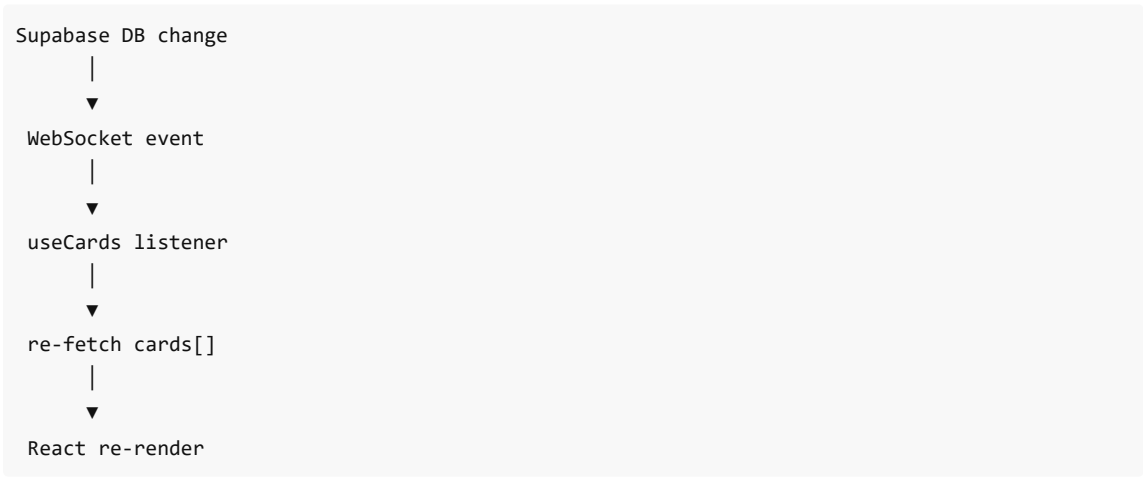
Column IDs

id	Label	Color
`not-started`	Not Started	Gray
`wip`	WIP	Blue — **limit: 6**
`recurring`	Recurring	Purple
`completed`	Completed	Green
`deferred`	Deferred / Blocked	Coral/Red
`future`	Future Pick	Amber

Architecture Overview



Realtime Loop



File-by-File Breakdown

Root / Config

`index.html`

The single HTML page Vite serves. Contains only a `<div id="root">` and a `<script type="module">` pointing at `src/main.jsx`. Vite injects the bundled output here at build time.

`vite.config.js`

```
export default defineConfig({ plugins: [react()] })
```

Minimal config. Loads the `@vitejs/plugin-react` plugin which enables:

- Fast Refresh (hot reload without losing state)
- JSX transform (no need to `import React` in every file)

`package.json`

Defines three scripts:

- `npm run dev` → starts the Vite dev server (localhost:5173)
- `npm run build` → bundles to `dist/`
- `npm run preview` → serves the built `dist/` locally

Four runtime dependencies (`react` , `react-dom` , `@hello-pangea/dnd` , `@supabase/supabase-js`) and two dev dependencies (`vite` , `@vitejs/plugin-react`).

`.env.example`

Template showing the two required environment variables:

- `VITE_SUPABASE_URL` — your project's Supabase REST endpoint
- `VITE_SUPABASE_ANON_KEY` — the public anon key for client-side auth

Copy to `.env` and fill in real values. The `VITE_` prefix makes Vite expose these to client-side code via `import.meta.env` .

Entry Point

`src/main.jsx`

```
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode><App /></React.StrictMode>
)
```

The React bootstrap. Mounts `App` into the `#root` div. `React.StrictMode` runs effects twice in development to surface side-effect bugs.

App Layer

`src/constants.js`

The single source of truth for all design tokens and configuration.

- **`WIP_LIMIT = 6`** — maximum cards allowed in the WIP column simultaneously.
- **`COLUMNS`** — array of 6 column descriptors. Each has `{ id, label, color, limit? }` . Drives everything: the board layout, column headers, badge colors.
- **`COLORS`** — maps each color name (`gray` , `blue` , `purple` , `green` , `coral` , `amber`) to a full set of tokens: `columnBg` , `headerBg` , `headerText` , `accent` , `badgeBg` , `badgeText` . Used by `Column` and `Card` for dynamic inline styles.

- `PRIORITY_COLORS` — maps `high` / `medium` / `low` to `{ bg, text }` badge color pairs.

Everything visual derives from this file. Changing a color here propagates everywhere.

`src/App.jsx`

The root component. Owns all UI state above the data layer.

State it manages:

- `search` (string) — live text filter against card titles
- `priorityFilter` (string) — selected priority badge filter
- `dueFilter` (string) — selected due-date range filter

Derived state via `useMemo`:

- `filteredCards` — cards array after applying all three filters
- `cardsByCol` — `{ [colId]: Card[] }` map, sorted by `position`, built from `filteredCards`. This is what `Board` receives.
- `stats` — `{ total, wip, overdue, completed }` computed from the *unfiltered* `cards` array (stats always reflect reality, not current filter).

`getDueStatus(due_date)` — utility defined at module level. Returns `'overdue'`, `'this-week'`, `'later'`, or `'none'` based on today's date. Used both here (for filter logic) and in `Card.jsx` (for badge display).

Renders:

```
<header>        ← title + search input
<StatsBar>      ← numbers strip
<FilterBar>     ← filter buttons
<Board>         ← the kanban grid
```

State & Data Layer

`src/lib/supabase.js`

```
export const supabase = createClient(VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY)
```

Creates and exports a single Supabase client instance. Imported wherever Supabase is needed (currently only `useCards.js`). Using a singleton ensures only one WebSocket connection is opened.

`src/hooks/useCards.js`

The data backbone of the entire app. A custom React hook that:

1. **Initial fetch** (`useEffect` on mount):
   Queries `cards` table ordered by `position`, sets state.
2. **Realtime subscription** (same `useEffect`):
   Opens a Supabase Realtime channel listening for any `INSERT`, `UPDATE`, or `DELETE` on `public.cards`.
   On any change it re-fetches the full list. This means two browser tabs stay in sync automatically.
3. **Cleanup**: Unsubscribes the channel and sets `mounted = false` when the component unmounts, preventing state updates on dead components.

Exposed functions:

Function	What it does
`addCard({ title, priority, due_date, col })`	Inserts a row; sets `position` to current column length; updates local state on success
`deleteCard(id)`	Optimistic local remove, then `DELETE` in Supabase
`moveCard({ srcColId, dstColId, newSrcList, newDstList })`	**Optimistic update**: immediately rebuilds the cards map with new `col` + `position` values, then fires parallel Supabase `UPDATE` calls for every affected card

The **optimistic update** in `moveCard` is the most complex piece: it maps over all cards, patches only the ones in the affected columns, and returns the new state before the server confirms — making drag-and-drop feel instant.

Components

`src/components/Board.jsx`

Wraps the entire column grid in `@hello-pangea/dnd`'s `<DragDropContext>`.

State: `draggingFromCol` (string | null) — set on `onDragStart`, cleared on `onDragEnd`. Passed down to every `Column` so the WIP column knows whether to accept a drop.

`onDragEnd(result)`:

1. Bail early if no destination or position unchanged.
2. Guard: if destination is `wip` and source is not `wip` and WIP is at limit → abort.
3. Splice the moved card out of the source list.
4. If same column: reorder in place. If different column: insert into destination list with updated `col`.
5. Call `moveCard(...)` from `useCards`.

Renders `<Column>` for each entry in `COLUMNS`, passing `cards={cardsByCol[col.id]}`.

`src/components/Column.jsx`

Renders one kanban column as a `<Droppable>` zone.

WIP enforcement (two layers):

- `isDropDisabled` — set to `true` when the destination is WIP, the card is coming from another column, and WIP is full. The `@hello-pangea/dnd` library respects this and shows a blocked cursor.
- Visual `wip-banner` — shown when the column is at capacity.
- `+` button `disabled` — when `wipFull`, the add button is grayed out.

The column header shows `4/6` style count for WIP and plain count for all others.

Clicking `+` sets `showModal = true` → renders `<AddCardModal>` inline.

`src/components/Card.jsx`

Renders one card as a `<Draggable>`.

Due date badge logic:

- `getDueStatus()` returns one of: `overdue` (red), `this-week` (yellow), `later` (slate), or `null` (no badge).
- Cards in the `completed` column get a strikethrough on their title.
- The left border color matches the column's accent color, passed in as `accentColor` prop.

Drag visual: `snapshot.isDragging` applies `.card-dragging` class — drop shadow + 1° rotation.

Delete button uses `e.stopPropagation()` so clicking it doesn't accidentally start a drag.

`src/components/AddCardModal.jsx`

A form modal for creating a new card.

Local state: `title`, `priority` (default `'medium'`), `dueDate`, `saving`, `error`.

Submit flow:

1. Validate title is not empty.
2. Set `saving = true`, call `addCard(...)`.
3. On success → `onClose()`. On error → display `error.message`.

Backdrop click closes the modal via `onClick={onClose}`. The inner modal div stops propagation so clicking inside doesn't close it.

The primary button inherits the column's accent color via `style={{ backgroundColor: colors.accent }}`.

`src/components/FilterBar.jsx`

A stateless UI component. Renders two groups of pill buttons:

- **Priority group:** All Priority / High / Medium / Low
- **Due date group:** All Dates / Overdue / This Week / Later

Active state is purely driven by the `priorityFilter` and `dueFilter` props from `App`. Clicking a button calls `onPriorityChange` or `onDueChange` with the new value (or empty string to clear).

`src/components/StatsBar.jsx`

Purely presentational. Receives a `stats` object and renders four numbers:

- **Total** (neutral) — all cards across all columns
- **In Progress** (blue) — cards in the `wip` column
- **Overdue** (red) — cards with a past due date
- **Completed** (green) — cards in the `completed` column

Stats are always computed from unfiltered data so they reflect the true board state.

Styling

`src/App.css`

A single flat CSS file (no CSS modules, no Tailwind). Organized into sections:

- **Reset** — `box-sizing: border-box`, zero margins/padding
- **Body** — system font stack, slate background `#F1F5F9`
- **Header** — dark `#1E293B` bar with frosted-glass search input
- **Stats Bar** — white strip, bold stat numbers
- **Filter Bar** — pill buttons with active state

- **Board** — horizontal flex with `overflow-x: auto` for many columns
- **Column** — fixed 268px width, max-height with inner scroll
- **Card** — white card, left border accent, hover shadow, drag shadow
- **Badge** — tiny uppercase pill used for priority and due date
- **Modal** — fixed overlay with `backdrop-filter: blur(3px)`
- **Scrollbars** — custom 5px slim scrollbar via `-webkit-scrollbar`

All color values are hardcoded from the Tailwind color palette (no Tailwind dependency — just the values).

Data Flow

```
User action
    │
    ▼
Component event handler
    │
    ├─► addCard / deleteCard / moveCard  (from useCards)
    │         │
    │         ├─ Optimistic: setCards(...) immediately
    │         └─ Async: supabase.from('cards')...
    │
    └─► setSearch / setPriorityFilter / setDueFilter  (in App)
              │
              └─ useMemo recomputes filteredCards + cardsByCol
                        │
                        └─ Board re-renders columns
```

```
Supabase DB (any tab / external mutation)
    │
    ▼
Realtime WebSocket event
    │
    ▼
useCards channel listener
    │
    ▼
Re-fetch + setCards(data)
    │
    ▼
React re-render (all tabs)
```

Key Design Decisions

Decision	Why
No global state library	The app has one data source (cards[]) and one owner (useCards). Lifting state to App + passing props is sufficient. Adding Redux/Zustand would be over-engineering.

Optimistic updates in moveCard	Drag-and-drop must feel instant. Waiting for a Supabase round-trip would cause visible lag and snap-back.
Realtime via re-fetch, not patch	Simpler and more robust. A WebSocket event triggers a full re-fetch rather than trying to patch local state from the event payload, avoiding edge cases.
Single CSS file	The project is small enough that CSS modules or Tailwind adds complexity without benefit. One flat file is easy to scan and override.
WIP limit enforced at two layers	`isDropDisabled` on the Droppable (prevents the drop itself) + the guard in `onDragEnd` (safety net). Defense in depth.
`getDueStatus` duplicated in App and Card	Intentional. The App version drives filter logic; the Card version drives display. Sharing via import would couple unrelated concerns.
`position` as an integer	Cards are ordered by position within their column. On add, position = current column length. On move, all affected cards get re-numbered 0…N. No fractional indexing.
Supabase anon key in client	Expected for Supabase. Row-level security (RLS) policies on the Supabase side control access. The anon key is safe to expose publicly.

Environment Variables

```
VITE_SUPABASE_URL=https://your-project-id.supabase.co
VITE_SUPABASE_ANON_KEY=your-anon-key-here
```

Copy `.env.example` to `.env` in the project root. Never commit `.env` — it's in `.gitignore`.

These are exposed to client-side code via `import.meta.env.VITE_*`. Vite strips any env var that does NOT start with `VITE_` from the client bundle as a security measure.